Accessible Voice Launcher Architecture

Complete Technical Blueprint: Gemini NLU, System Automation & Android Accessibility API

Target OS: Android 12+ (API 31+) • Language: Kotlin • AI Architecture: Gemini Function Calling

1. Architecture & System Blueprint

Building a custom accessible launcher for elderly users or individuals with motor disabilities requires a multi-layered architecture. Rather than relying on simple voice commands, this system integrates **Gemini NLU (Tool Calling)** as an orchestration engine, translating intent into native Android API execution.

Component Module	Android Subsystem / API	Role & Operational Scope
Gemini Assistant Engine	Gemini API `generativeai`	Parses spoken user input into structured JSON tool parameters via Function Calling.
Contacts Resolver	ContentResolver `ContactsContract`	Resolves spoken contact names or fuzzy search terms to valid telephone numbers.
WhatsApp Inline Engine	NotificationListener `RemoteInput`	Intercepts incoming notifications and injects replies directly into system intents.
Accessibility UI Fallback	AccessibilityService `AccessibilityNodeInfo`	Automates UI clicks, text injection, and navigation when native APIs are unavailable.
Emergency SOS Service	Foreground Service `FusedLocation`	Dispatches high-priority SMS with geolocation and initiates speakerphone calls.

> **Key Operational Flow**
>
> 1. User speaks a command (e.g., *"Manda un messaggio a Marco dicendo che sto arrivando"*).
> 2. Gemini parses the prompt and issues a function call to `send_whatsapp` or `send_sms` with target name and message.
> 3. The `ContactsResolver` maps *"Marco"* to a phone number via `ContactsContract`.
> 4. The app attempts delivery via `RemoteInput` (if active notification exists) or falls back to `AccessibilityService` UI injection.

2. Gemini NLU & Native Function Execution

The `GeminiAssistantManager` uses `generativeai:0.9.0` to bind native functions (SMS, WhatsApp, SOS) directly to the LLM context. Structured responses guarantee robust execution.

```kotlin
package com.accessibility.launcher.ai

import android.content.Context
import android.telephony.SmsManager
import com.google.ai.client.generativeai.GenerativeModel
import com.google.ai.client.generativeai.type.FunctionResponsePart
import com.google.ai.client.generativeai.type.Schema
import com.google.ai.client.generativeai.type.Tool
import com.google.ai.client.generativeai.type.content
import com.google.ai.client.generativeai.type.defineFunction
import kotlinx.serialization.json.*

/**
 * Executes system-level native capabilities triggered by Gemini NLU function calls.
 */
class NativeActionExecutor(private val context: Context) {

    fun sendSms(phoneNumber: String, message: String): JsonObject {
        return try {
            val smsManager = context.getSystemService(SmsManager::class.java)
            smsManager.sendTextMessage(phoneNumber, null, message, null, null)
            buildJsonObject {
                put("status", JsonPrimitive("success"))
                put("details", JsonPrimitive("SMS successfully dispatched to $phoneNumber"))
            }
        } catch (e: Exception) {
            buildJsonObject {
                put("status", JsonPrimitive("error"))
                put("reason", JsonPrimitive(e.localizedMessage ?: "SMS execution failed"))
            }
        }
    }

    fun triggerEmergencySos(): JsonObject {
        // Dispatches emergency routine (location grab, emergency contact SMS, acoustic alarm)
        return buildJsonObject {
            put("status", JsonPrimitive("success"))
            put("action", JsonPrimitive("SOS sequence initiated"))
        }
    }
}

/**
 * Orchestrates NLU via Gemini Function Calling and manages execution loops.
 */
class GeminiAssistantManager(
    apiKey: String,
    private val actionExecutor: NativeActionExecutor
) {
    private val sendSmsFunction = defineFunction(
        name = "send_sms",
        description = "Sends an SMS text message to a specified contact number.",
        parameters = listOf(
            Schema.str("phoneNumber", "Target phone number in international format."),
            Schema.str("message", "Text payload of the message.")
        ),
        requiredParameters = listOf("phoneNumber", "message")
    )

    private val triggerSosFunction = defineFunction(
        name = "trigger_sos",
        description = "Triggers the emergency SOS routine for immediate medical/safety help.",
```

```
        parameters = emptyList(),
        requiredParameters = emptyList()
    )

    private val model = GenerativeModel(
        modelName = "gemini-2.5-flash",
        apiKey = apiKey,
        tools = listOf(Tool(listOf(sendSmsFunction, triggerSosFunction)))
    )

    suspend fun processIntent(userSpeechInput: String): String {
        val chat = model.startChat()
        var response = chat.sendMessage(
            content { text("User Voice Input: $userSpeechInput") }
        )

        while (true) {
            val functionCall = response.functionCalls.firstOrNull() ?: break

            val executionResultJson = when (functionCall.name) {
                "send_sms" -> {
                    val phone = functionCall.args["phoneNumber"]?.jsonPrimitive?.content.orEmpty
()
                    val text = functionCall.args["message"]?.jsonPrimitive?.content.orEmpty()
                    actionExecutor.sendSms(phone, text)
                }
                "trigger_sos" -> actionExecutor.triggerEmergencySos()
                else -> buildJsonObject { put("status", JsonPrimitive("unsupported_tool")) }
            }

            response = chat.sendMessage(
                content("user") {
                    part(FunctionResponsePart(functionCall.name, executionResultJson))
                }
            )
        }

        return response.text ?: "Operazione completata."
    }
}
```

3. Contacts Resolution via ContactsContract API

When users dictate names (e.g., *"Mamma"*, *"Giuseppe Rossi"*), the launcher must resolve them to phone numbers using Android's `ContentResolver`. The following implementation queries `ContactsContract.CommonDataKinds.Phone` with sanitized fuzzy matching.

```kotlin
package com.accessibility.launcher.contacts

import android.content.Context
import android.database.Cursor
import android.provider.ContactsContract
import java.text.Normalizer

data class ContactMatch(
    val displayName: String,
    val phoneNumber: String,
    val confidenceScore: Float
)

class ContactsResolver(private val context: Context) {

    /**
     * Resolves a spoken contact query string to the most relevant phone number.
     */
    fun resolveContactNumber(spokenQuery: String): ContactMatch? {
        val normalizedQuery = normalizeString(spokenQuery)
        val matches = mutableListOf<ContactMatch>()

        val projection = arrayOf(
            ContactsContract.CommonDataKinds.Phone.DISPLAY_NAME_PRIMARY,
            ContactsContract.CommonDataKinds.Phone.NUMBER,
            ContactsContract.CommonDataKinds.Phone.IS_SUPER_PRIMARY
        )

        val cursor: Cursor? = context.contentResolver.query(
            ContactsContract.CommonDataKinds.Phone.CONTENT_URI,
            projection,
            null,
            null,
            "${ContactsContract.CommonDataKinds.Phone.IS_SUPER_PRIMARY} DESC"
        )

        cursor?.use { c ->
            val nameIdx =
c.getColumnIndex(ContactsContract.CommonDataKinds.Phone.DISPLAY_NAME_PRIMARY)
            val numberIdx = c.getColumnIndex(ContactsContract.CommonDataKinds.Phone.NUMBER)

            while (c.moveToNext()) {
                val name = c.getString(nameIdx) ?: continue
                val number = c.getString(numberIdx) ?: continue

                val normalizedName = normalizeString(name)
                val score = calculateMatchScore(normalizedQuery, normalizedName)

                if (score > 0.4f) {
                    val cleanNumber = number.replace("[^0-9+]".toRegex(), "")
                    matches.add(ContactMatch(name, cleanNumber, score))
                }
            }
        }

        return matches.maxByOrNull { it.confidenceScore }
    }

    private fun normalizeString(input: String): String {
        val temp = Normalizer.normalize(input, Normalizer.Form.NFD)
        return temp.replace("\p{InCombiningDiacriticalMarks}+".toRegex(), "")
            .lowercase()
```

```
            .trim()
    }

    private fun calculateMatchScore(query: String, target: String): Float {
        if (query == target) return 1.0f
        if (target.contains(query) || query.contains(target)) return 0.8f
        return 0.0f
    }
}
```

4. WhatsApp Interception & RemoteInput Inline Reply

For WhatsApp, the preferred method avoids launching the foreground app. A `NotificationListenerService` captures incoming notifications and dispatches replies using system `RemoteInput` bundles.

```kotlin
package com.accessibility.launcher.notifications

import android.app.Notification
import android.app.RemoteInput
import android.content.Intent
import android.os.Bundle
import android.service.notification.NotificationListenerService
import android.service.notification.StatusBarNotification

class WhatsAppNotificationService : NotificationListenerService() {

    override fun onNotificationPosted(sbn: StatusBarNotification) {
        if (sbn.packageName != "com.whatsapp") return

        val notification = sbn.notification ?: return
        val extras = notification.extras ?: return

        val sender = extras.getCharSequence(Notification.EXTRA_TITLE)?.toString() ?: "Unknown"
        val text = extras.getCharSequence(Notification.EXTRA_TEXT)?.toString() ?: ""

        val replyAction = findInlineReplyAction(notification) ?: return
        onMessageCaptured(sender, text, replyAction)
    }

    private fun findInlineReplyAction(notification: Notification): Notification.Action? {
        return notification.actions?.firstOrNull { action ->
            action.remoteInputs?.any { it.allowFreeFormInput } == true
        }
    }

    fun sendReply(replyAction: Notification.Action, replyText: String): Boolean {
        val remoteInputs = replyAction.remoteInputs ?: return false
        val intent = Intent()
        val bundle = Bundle()

        for (remoteInput in remoteInputs) {
            bundle.putCharSequence(remoteInput.resultKey, replyText)
        }

        RemoteInput.addResultsToIntent(remoteInputs, intent, bundle)

        return try {
            replyAction.actionIntent.send(this, 0, intent)
            true
        } catch (e: Exception) {
            false
        }
    }

    private fun onMessageCaptured(sender: String, message: String, action: Notification.Action)
{
        // Send payload to TTS engine and store reply handle for Gemini execution
    }
}
```

5. AccessibilityService UI Fallback Automation

If no active notification exists, the system launches WhatsApp via a deep link and uses an `AccessibilityService` to inject text into `com.whatsapp:id/entry` and click `com.whatsapp:id/send`.

```kotlin
package com.accessibility.launcher.services

import android.accessibilityservice.AccessibilityService
import android.content.Intent
import android.net.Uri
import android.os.Bundle
import android.view.accessibility.AccessibilityEvent
import android.view.accessibility.AccessibilityNodeInfo

class WhatsAppAccessibilityFallbackService : AccessibilityService() {

    companion object {
        private const val ID_ENTRY = "com.whatsapp:id/entry"
        private const val ID_SEND = "com.whatsapp:id/send"

        @Volatile var pendingMessage: String? = null
        @Volatile var isAutomationActive: Boolean = false

        fun launchFallback(context: AccessibilityService, phone: String, message: String) {
            pendingMessage = message
            isAutomationActive = true
            val intent = Intent(Intent.ACTION_VIEW).apply {
                data = Uri.parse("https://api.whatsapp.com/send?phone=$phone")
                addFlags(Intent.FLAG_ACTIVITY_NEW_TASK or Intent.FLAG_ACTIVITY_CLEAR_TOP)
            }
            context.startActivity(intent)
        }
    }

    override fun onAccessibilityEvent(event: AccessibilityEvent?) {
        if (!isAutomationActive || event?.packageName != "com.whatsapp") return
        val rootNode = rootInActiveWindow ?: return

        val message = pendingMessage ?: return
        val inputNodes = rootNode.findAccessibilityNodeInfosByViewId(ID_ENTRY)

        if (!inputNodes.isNullOrEmpty()) {
            val args = Bundle().apply {
                putCharSequence(AccessibilityNodeInfo.ACTION_ARGUMENT_SET_TEXT_CHARSEQUENCE,
message)
            }
            if (inputNodes[0].performAction(AccessibilityNodeInfo.ACTION_SET_TEXT, args)) {
                val sendNodes = rootNode.findAccessibilityNodeInfosByViewId(ID_SEND)
                if (!sendNodes.isNullOrEmpty() && sendNodes[0].performAction(AccessibilityNodeI
nfo.ACTION_CLICK)) {
                    isAutomationActive = false
                    pendingMessage = null
                    performGlobalAction(GLOBAL_ACTION_HOME)
                }
            }
        }
    }

    override fun onInterrupt() { isAutomationActive = false }
}
```

6. System Manifest & Permissions Configuration

The Android Manifest must register all specialized services and declare sensitive permissions required for full device control and accessibility support.

```xml
<!-- AndroidManifest.xml -->
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.accessibility.launcher">

    <uses-permission android:name="android.permission.SEND_SMS" />
    <uses-permission android:name="android.permission.READ_CONTACTS" />
    <uses-permission android:name="android.permission.CALL_PHONE" />
    <uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />
    <uses-permission android:name="android.permission.RECORD_AUDIO" />

    <application
        android:label="Accessible AI Launcher"
        android:theme="@style/Theme.Material3.DayNight.NoActionBar">

        <!-- Notification Listener Service -->
        <service android:name=".notifications.WhatsAppNotificationService"
            android:permission="android.permission.BIND_NOTIFICATION_LISTENER_SERVICE"
            android:exported="true">
            <intent-filter>
                <action android:name="android.service.notification.NotificationListenerService"
/>
            </intent-filter>
        </service>

        <!-- Accessibility Fallback Service -->
        <service android:name=".services.WhatsAppAccessibilityFallbackService"
            android:permission="android.permission.BIND_ACCESSIBILITY_SERVICE"
            android:exported="true">
            <intent-filter>
                <action android:name="android.accessibilityservice.AccessibilityService" />
            </intent-filter>
            <meta-data
                android:name="android.accessibilityservice"
                android:resource="@xml/whatsapp_accessibility_config" />
        </service>

    </application>
</manifest>
```